

Gramática de la práctica 4

Kevin Isaac Alcántara Estrada

October 2024

1 Identificar elementos de la gramática

Primero debes tomar en cuenta las expresiones regulares que deben ser identificadas por el autómata.

1.1 Producciones

Gramática P

$P = \{$

S	$\rightarrow$	$\text{Expr} \mid \text{Asig}$
Expr	$\rightarrow$	$\text{Term Expr}'$
Expr'	$\rightarrow$	$+ \text{Term Expr}' \mid - \text{Term Expr}' \mid \epsilon$
Term	$\rightarrow$	$\text{Factor Term}'$
Term'	$\rightarrow$	$* \text{Factor Term}' \mid / \text{Factor Term}' \mid \epsilon$
Factor	$\rightarrow$	$\text{Num} \mid \text{Var} \mid (\text{Expr}) \mid - \text{Expr}$
Num	$\rightarrow$	Entero Decimal
Decimal	$\rightarrow$	$. \text{Entero} \mid \epsilon$
Entero	$\rightarrow$	$\text{Digito} \mid \text{Digito Entero}$
Digito	$\rightarrow$	$0 \mid 1 \mid 2 \mid \dots \mid 9$
Asig	$\rightarrow$	$\text{var Var} = \text{Expr}$
Var	$\rightarrow$	Letra Pos
Pos	$\rightarrow$	$\text{Var} \mid \epsilon$
Letra	$\rightarrow$	$- \mid a \mid b \mid \dots \mid z \mid A \mid B \mid \dots \mid Z$

$\}$

1.2 Símbolos no terminales

El conjunto de símbolos no terminales N es el siguiente:

$N = \{S, \text{Expr}, \text{Expr}', \text{Term}, \text{Term}', \text{Factor}, \text{Num}, \text{Decimal}, \text{Entero}, \text{Digito}, \text{Asig}, \text{Var}, \text{Pos}, \text{Letra} \}$

1.3 Símbolos terminales

El conjunto de símbolos terminales Σ es el siguiente:

(se usó la notación para rangos para ahorrar espacio)

$\Sigma = \{ \text{"var"}, [0-9], [_aZ-AZ], \text{"("}, \text{")"}, \text{"+"}, \text{"-"}, \text{"*"}, \text{"/"}, \text{"="} \}$

1.4 Símbolos inicial

Nuestro símbolo inicial $S \in N$ para la gramática sería $S = S$

2 Eliminación de la ambigüedad en la gramática

Podemos ver que la gramática no es ambigua porque no tiene más de una producción del estilo $S \rightarrow S\alpha S$. Además, no cuenta tampoco con "ciclos" en las producciones, por ello, podemos decir que la gramática no es ambigua y no requiere procesamiento para la misma.

3 Eliminar recursividad izquierda de la gramática

Podemos notar que no existe en la gramática una producción donde el encabezado y el primer elemento del cuerpo de la producción sean el mismo símbolo no terminal, por lo tanto, no tenemos en la gramática recursividad izquierda y no se necesita procesamiento para la misma.

4 Factorización izquierda de la gramática

En este caso, no es necesario realizar una factorización izquierda de la gramática debido a que no existe más de una producción tal que para algún $A \in N$ ocurra que $A \rightarrow \alpha\beta_1|\alpha\beta_2|...|\alpha\beta_i|\gamma_1|\gamma_2|...|\gamma_k$

5 Eliminación de conflictos shift/reduce

Una vez hemos probado la gramática actual en código, la herramienta yacc no detectó conflictos del tipo shift/reduce, por lo que no fue necesario llevar a cabo otro procesamiento a nuestra gramática.

6 Nuevos conjuntos N y P

Nuestro conjunto de producciones quedó exactamente igual.

```
P = {
  Expr    → Term Expr'
  Expr'   → + Term Expr' | - Term Expr' | ε
  Term    → Factor Term'
  Term'   → * Factor Term' | / Factor Term' | ε
  Factor  → Num | Var | ( Expr ) | - Expr
  Num     → Entero Decimal
  Decimal → . Entero | ε
  Entero  → Digito | Digito Entero
  Digito  → 0 | 1 | 2 | ... | 9
  Asig    → var Var = Expr
  Var     → Letra Pos
  Pos     → Var | ε
  Letra   → _ | a | b | ... | z | A | B | ... | Z
}
```

El conjunto de símbolos no terminales no sufrió cambios y es:

$N = \{S, Expr, Expr', Term, Term', Factor, Num, Decimal, Entero, Digito, Asig, Var, Pos, Letra\}$